

Laborator nr. 7

Interogări de tip ”JOIN” în MongoDB

Titular disciplină
șef lucr. dr. ing. Cristian Aflori
cristian.aflori@academic.tuiasi.ro

Titular aplicații
șef lucr. dr. ing. Cătălin Mironeanu
catalin.mironeanu@academic.tuiasi.ro

Titular aplicații, ing. Elena-Denisa Chelarasu
elena-denisa.chelarasu@academic.tuiasi.ro

Titular aplicații, ing. Ioana-Cristina Prioteasa
ioana-cristina.prioteasa@academic.tuiasi.ro

an universitar 2025 - 2026



Obiective:

- Operatorul \$lookup și tipuri de Join
- Condiții de ”join” și subinterogari pe colecții de tip ”joined”
- Subinterogari corelate folosind sintaxa concisă

1 Operatorul \$lookup și tipuri de Join

În MongoDB, pentru a interoga documente din două colecții din aceeași bază de date, se folosește operatorul \$lookup al *framework*-ului aggregate [1]. Operatorul \$lookup adaugă un atribut de tip vector pentru fiecare document de intrare. Câmpul nou de tip vector (*array*) conține toate documentele care se potrivesc din colecția externă (*”joined collection”*). Astfel, vectorul cu noile documente sunt disponibile în următoarea etapă (*”stage”*) din *aggregate*.

Operatorul \$lookup reprezintă o etapă distinctă în cadrul instrucțiunii *aggregate*, iar sintaxa de baza este următoarea:

```
{
  $lookup:
  {
    from: <collection to join>,
    localField: <field from the input documents>,
    foreignField: <field from the documents of the "from" collection>,
    as: <output array field>
  }
}
```

Atributele principale ale operatorului \$lookup (referința bibliografică [2]):

- *from*: specifică colecția cu care se face ”join”
- *localField*: specifică acele câmpuri din colecția de bază ale căror valori sunt egale cu valorile din câmpurile specificate în *foreignField* din colecția cu care se face ”join”
- *foreignField*: specifică câmpul din documentele din colecția de ”join” care au valori egale cu valorile din câmpul local din documentele de intrare. Dacă un document din colecția de la *from* nu conține valori corespunzătoare în câmpul străin, operatorul \$lookup tratează valoarea ca null.
- *as*: specifică numele noului câmp de tip vector ce conține documentele care se potrivesc din colecția de la *from*. Dacă numele specificat există deja în documentul de intrare, câmpul existent este suprascris.

Instrucțiunea corespunzătoare din limbajul SQL este:

```
1 SELECT *, <output array field>
2 FROM collection
3 WHERE <output array field> IN (
4   SELECT *
5   FROM <collection to join>
6   WHERE <foreignField> = <collection.localField>
7 );
```

1.1 Exemplu de folosire a operatorului \$lookup pentru operații de “join” între 2 colecții bazate pe o relație de egalitate între valori

Se dau două colecții, orders ce conține comenzile făcute la un moment dat și colecția inventory ce conține produsele disponibile pentru comenzi.

```
db.orders.insertMany( [
  { "_id": 1, "item": "almonds", "price": 12, "quantity": 2 },
  { "_id": 2, "item": "pecans", "price": 20, "quantity": 1 },
  { "_id": 3, "item": "bread" }
] )

db.inventory.insertMany( [
  { "_id": 1, "sku": "almonds", "description": "product 1", "instock": 120 },
  { "_id": 2, "sku": "bread", "description": "product 2", "instock": 80 },
  { "_id": 3, "sku": "cashews", "description": "product 3", "instock": 60 },
  { "_id": 4, "sku": "pecans", "description": "product 4", "instock": 70 },
  { "_id": 5, "sku": null, "description": "Incomplete" },
  { "_id": 6 }
] )
```

Pentru a afișa valorile din inventory pentru produsele din orders, trebuie aplicat operatorul \$lookup pe câmpurile item (“localField”) și sku (“foreignField”):

```
db.orders.aggregate( [
  {
    $lookup:
    {
      from: "inventory",
      localField: "item",
      foreignField: "sku",
      as: "inventory_docs"
    }
  }
] )
```

Instrucțiunea SQL corespunzătoare este:

```
SELECT *, inventory_docs
FROM orders
WHERE inventory_docs IN (
  SELECT *
  FROM inventory
  WHERE sku = orders.item
);
```

De asemenea, se poate folosi operatorul \$mergeObjects pentru a combina documente multiple într-un singur document.

```
db.orders.aggregate( [
  {
    $lookup: {
      from: "inventory",
      localField: "item",      // field in the orders collection
      foreignField: "sku",    // field in the items collection
      as: "fromItems"
    }
  },
  {
    $replaceRoot: { newRoot: { $mergeObjects: [ { $arrayElemAt: [ "$fromItems", 0 ] }, "$$ROOT" ] } }
  },
  { $project: { fromItems: 0 } }
] )
```

1.2 Exemplu \$lookup pentru câmpuri de tip vector

Câmpul specificat în clauza “*localField*” poate fi scalar sau poate fi de tip vector. În exemplul anterior, câmpul este scalar. Se poate face “join” și cu atributul de tip vector, fără a fi nevoie să aplicăm într-o etapă anterioară operatorul *unwind*.

Fie două colecții, una de Clase (colecția *classes*) și una de Membri (colecția *members*), astfel:

```
db.classes.insertMany( [
  { _id: 1, "title": "Reading is ...", "enrollmentlist": [ "giraffe2", "pandabear", "artie" ],
    "days": [ "M", "W", "F" ] },
  { _id: 2, "title": "But Writing ...", "enrollmentlist": [ "giraffel", "artie" ],
    "days": [ "T", "F" ] }
] )

db.members.insertMany( [
  { _id: 1, "name": "artie", "joined": new Date("2016-05-01"), "status": "A" },
  { _id: 2, "name": "giraffe", "joined": new Date("2017-05-01"), "status": "D" },
  { _id: 3, "name": "giraffel", "joined": new Date("2017-10-01"), "status": "A" },
  { _id: 4, "name": "panda", "joined": new Date("2018-10-11"), "status": "A" },
  { _id: 5, "name": "pandabear", "joined": new Date("2018-12-01"), "status": "A" },
  { _id: 6, "name": "giraffe2", "joined": new Date("2018-12-01"), "status": "D" }
] )
```

Pentru a afla și informațiile despre fiecare membru înrolat la un curs se folosește \$lookup:

```
db.classes.aggregate( [
  { $lookup: {
    from: "members",
    localField: "enrollmentlist",
    foreignField: "name",
    as: "enrollee_info"
  } }
] )
```

2 Condiții de “join” și subinterogări pe colecții de tip “joined”

MongoDB suportă următoarele operații de tip “join”:

- operații de tip *pipeline* bazate pe relații de egalitate între valorile câmpurilor;
- condiții multiple de “join”;
- subinterogări necorelate și subinterogări corelate.

O interogare corelată în MongoDB reprezintă o etapă în procesul de *pipeline* a operatorului \$lookup care referă documente din colecția externă (“*joined collection*”). O interogare necorelată nu face referire la câmpurile din colecția externă.

Interogările corelate din MongoDB sunt echivalente cu interogările corelate din SQL, unde subinterogarea referă valori din interogarea externă.

Pentru a executa interogări corelate sau necorelate pe 2 colecții și pentru a adăuga și alte condiții de “join” în afara de cele bazate pe relația de egalitate, se folosește următoarea sintaxă:

```
{
  $lookup:
  {
    from: <joined collection>,
    let: { <var_1>: <expression>, ..., <var_n>: <expression> },
    pipeline: [ <pipeline to run on joined collection> ],
    as: <output array field>
  }
}
```

- *from*: specifică colecția cu care se face “join”;
- *let*: se definesc variabilele folosite în pipeline, pentru a accesa câmpurile din colecția externă;
- *pipeline*: operațiile care se execută pe colecția externă care determină ce documente vor fi returnate; pentru a returna toate documentele se specifică un *pipeline* vid []; *pipeline*-ul nu poate accesa direct documentele din colecția externă, pentru acest lucru se folosesc variabilele definite în clauza *let*;
- *as*: specifică numele noului câmp de tip vector care conține documentele care se potrivesc din colecția externă; dacă numele specificat există deja, câmpul existent este suprascris.

Instrucțiunea SQL echivalentă este:

```
1 SELECT *, <output array field>
2 FROM collection
3 WHERE <output array field> IN (
4     SELECT <documents as determined from the pipeline>
5     FROM <collection to join>
6     WHERE <pipeline>
7 );
```

2.1 Exemplu de interogare corelată cu includerea condițiilor de join multiple

Se dau două colecții: una cu depozite (warehouses) și una cu comenzi ale clienților (custOrders):

```
db.warehouses.insertMany( [
  { "_id": 1, "stock_item": "almonds", warehouse: "A", "instock": 120 },
  { "_id": 2, "stock_item": "pecans", warehouse: "A", "instock": 80 },
  { "_id": 3, "stock_item": "almonds", warehouse: "B", "instock": 60 },
  { "_id": 4, "stock_item": "cookies", warehouse: "B", "instock": 40 },
  { "_id": 5, "stock_item": "cookies", warehouse: "A", "instock": 80 }
] )

db.custOrders.insertMany( [
  { "_id": 1, "item": "almonds", "price": 12, "ordered": 2 },
  { "_id": 2, "item": "pecans", "price": 20, "ordered": 1 },
  { "_id": 3, "item": "cookies", "price": 10, "ordered": 60 }
] )
```

Vrem să aflăm pentru fiecare produs din comandă, depozitele care au stoc suficient pentru a putea fi onorate. *Pipeline*-ul are două operații:

1. se face o interogare corelată între `custOrders.item` și `warehouses.stock_item`;
2. se verifică condiția pentru cantități: `warehouses.instock ≥ custOrders.ordered`

```
db.custOrders.aggregate( [
  { $lookup: {
    from: "warehouses",
    let: { order_item: "$item", order_qty: "$ordered" },
    pipeline: [
      { $match:
        { $expr:
          { $and:
            [
              { $eq: [ "$stock_item", "$$order_item" ] },
              { $gte: [ "$instock", "$$order_qty" ] }
            ]
          }
        }
      },
      { $project: { stock_item: 0, _id: 0 } }
    ],
    as: "stockdata"
  }
] )
```

Instrucțiunea SQL echivalentă este:

```
SELECT *, stockdata
FROM orders
WHERE stockdata IN (
  SELECT warehouse, instock
  FROM warehouses
  WHERE stock_item = orders.item
  AND instock >= orders.ordered
);
```

2.2 Exemplu de interogare necorelată

Se dau două colecții: una cu absențele studenților (absences) și una cu vacanțe ale studenților (holidays):

```
db.absences.insertMany( [
  { "_id": 1, "student": "Ann Aardvark", "sickdays": [ new Date("2024-05-01"), new Date("2024-08-23") ] },
  { "_id": 2, "student": "Zoe Zebra", "sickdays": [ new Date("2024-02-01"), new Date("2024-05-23") ] },
] )

db.holidays.insertMany( [
  { "_id": 1, "year": 2024, "name": "New Years", "date": new Date("2024-01-01") },
  { "_id": 2, "year": 2024, "name": "Pi Day", "date": new Date("2024-03-14") },
  { "_id": 3, "year": 2024, "name": "Ice Cream Day", "date": new Date("2024-07-15") },
  { "_id": 4, "year": 2023, "name": "New Years", "date": new Date("2023-01-01") },
  { "_id": 5, "year": 2023, "name": "Ice Cream Day", "date": new Date("2023-07-16") }
] )
```

Să se afișeze pe anul 2024, pentru fiecare student, pe lângă absențe și numele și datele de vacanță:

```
db.absences.aggregate( [
  {
    $lookup:
    {
      from: "holidays",
      pipeline: [
        { $match: { year: 2024 } },
        { $project: { _id: 0, date: { name: "$name", date: "$date" } } },
        { $replaceRoot: { newRoot: "$date" } }
      ],
      as: "holidays"
    }
  }
] )
```

Instrucțiunea SQL echivalentă este:

```
SELECT *, holidays
FROM absences
WHERE holidays IN (
  SELECT name, date
  FROM holidays
  WHERE year = 2024
);
```

3 Subinterogări corelate cu sintaxa concisă

Începând cu MongoDB 5.0 se poate utiliza o sintaxă concisă pentru subinterogări corelate, în cadrul instrucțiunii fiind permisă accesarea directă a câmpurilor atât din colecția de bază cât și din colecția externă.

Sintaxa de bază este:

```
{
  $lookup:
  {
    from: <foreign collection>,
    localField: <field from local collection's documents>,
    foreignField: <|field from foreign collection's documents|>,
    let: { <var_1>: <expression>, ..., <var_n>: <expression> },
    pipeline: [ <pipeline to run> ],
    as: <output array field>
  }
}
```

Instrucțiunea SQL echivalentă este:

```
1 SELECT *, <output array field>
2 FROM localCollection
3 WHERE <output array field> IN (
4   SELECT <documents as determined from the pipeline>
5   FROM <foreignCollection>
6   WHERE <foreignCollection.foreignField> = <localCollection.localField>
7   AND <pipeline match condition>
8 );
```

3.1 Exemplu de interogare corelată folosind sintaxa concisă

Se dau două colecții: restaurante (restaurants) și comenzi clienți (clientOrders):

```
db.restaurants.insertMany( [
  {
    _id: 1,
    "name": "American Steak House",
    "food": [ "filet", "sirloin" ],
    "beverages": [ "beer", "wine" ]
  },
  {
    _id: 2,
    "name": "Honest John Pizza",
    "food": [ "cheese pizza", "pepperoni pizza" ],
    "beverages": [ "soda" ]
  }
] )

db.clientOrders.insertMany( [
  {
    _id: 1,
    "item": "filet",
    "restaurant_name": "American Steak House"
  },
  {
    _id: 2,
    "item": "cheese pizza",
    "restaurant_name": "Honest John Pizza",
    "drink": "lemonade"
  },
  {
    _id: 3,
    "item": "cheese pizza",
    "restaurant_name": "Honest John Pizza",
    "drink": "soda"
  }
] )
```

Afișați comenzile și restaurantele ("join" după clientOrders.restaurant_name și restaurants.name) în care băuturile comenzilor sunt incluse în meniurile restaurantelor ("join" după clientOrders.drink și restaurants.beverages).

Instrucțiunea cu sintaxa concisă pentru MongoDB cu versiunea minimă 5:

```
db.clientOrders.aggregate( [
  {
    $lookup: {
      from: "restaurants",
      localField: "restaurant_name",
      foreignField: "name",
      let: { orders_drink: "$drink" },
      pipeline: [ {
        $match: {
          $expr: { $in: [ "$$orders_drink", "$beverages" ] }
        }
      ] ,
      as: "matches"
    }
  }
] )
```

Instrucțiunea echivalentă fără sintaxa concisă:

```
db.clientOrders.aggregate( [
  {
    $lookup: {
      from: "restaurants",
      let: { orders_restaurant_name: "$restaurant_name",
            orders_drink: "$drink" },
      pipeline: [ {
        $match: {
          $expr: {
            $and: [
              { $eq: [ "$$orders_restaurant_name", "$name" ] },
              { $in: [ "$$orders_drink", "$beverages" ] }
            ]
          }
        }
      ] ,
      as: "matches"
    }
  }
] )
```

Instrucțiunea SQL echivalentă este:

```
SELECT *, matches
FROM clientOrders
WHERE matches IN (
  SELECT *
  FROM restaurants
  WHERE restaurants.name = clientOrders.restaurant_name
  AND restaurants.beverages = clientOrders.drink
);
```

4 Activități practice

Testați exemplele din laborator.

Creați următoarele colecții:

Discipline cu următoarea structură:

- denumire - cod disciplină
- evaluare - de tip subdocument cu modul de calculare a notei finale (ex: {partial: 0.4, examen: 0.4, proiect: 0.2})
- cerinte - de tip vector ce conține cerințele specifice disciplinei (ex: ["OOP", "programare web", "SQL", "MongoDB"])

Studenti cu următoarea structură:

- nrMatricol
- nume
- prenume
- grupa
- discipline - de tip vector ce conține codurile disciplinelor urmate de fiecare student
- skills - de tip vector ce conține abilitățile studentului (ex: ["OOP", "SQL", "MongoDB"])

ZileLibere cu următoarea structură:

- nume
- semestrul
- data

Catalog cu următoarea structură:

- cod - disciplina
- student - numărul matricol al studentului
- nota - nota la disciplină

Folosind colecțiile Discipline, Studenti, ZileLibere și Catalog:

1. Afișați pentru fiecare student și detaliile disciplinelor urmate.
2. Afișați pentru fiecare student zilele libere din semestrul al 2-lea.
3. Afișați disciplinele studenților cu detalii, unde capabilitățile studenților apar la cerințele disciplinelor.
4. Afișați notele studenților împreună cu detaliile acestora, ordonați descrescător după notă.

Uniți datele din catalog cu cele din studenți în același document.

Obs: Pentru interogările corelate folosiți ambele sintaxe: cea concisă și cea normală.

Bibliografie

- [1] “MongoDB Documentation - Aggregation Operations.”
<https://www.mongodb.com/docs/manual/aggregation/>.
- [2] “MongoDB Documentation - \$lookup (aggregation).”
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/lookup/>.